Customer Churn Analytics Project Document
S3 , Redshift

For task 1 and task 2 . Downloaded and extracted the file and uploaded inside raw folder of s3 bucket .

Task 3 — Create Amazon Redshift Cluster
Create a Redshift cluster with the following configuration:
Cluster type: Single node
Node type: dc2.large
Database name: telecom_dw
Configure an IAM role that allows Redshift to access S3

Updated the free tier version to get 300$ credit and to use the redshift service for free then after that created cluster but there was not option of dc2.large so chosed x3.large and cluster type single node and there was no option to put database name talecom_dw so continued without that .

After that created the iam permission of redshift access s3 and added that permission to that created cluster by adding redshiftS3access permission in permissions section .

Task 4 - Created table

```
CREATE TABLE stg_customer_churn (
    customer_id VARCHAR(20),
    gender VARCHAR(10),
    age INT,
    city VARCHAR(100),
    zip_code VARCHAR(10),
    tenure_in_months INT,
    monthly_charge DECIMAL(10,2),
    total_charges DECIMAL(12,2),
    customer_status VARCHAR(20)
);
```

```
CREATE TABLE stg_zip_population (
    zip_code VARCHAR(10),
    population INT
);
```

Created table in dev database .

Task 5 — Load Data into Redshift Use the COPY command to load the data from S3 into Redshift tables.
Data loading must follow best practices:
● Ignore headers ● Use CSV format ● Handle delimiters correctly

```
CREATE TABLE stg_zip_population (
    zip_code VARCHAR(10),
    population INT
);

SELECT current_database();


CREATE TABLE stg_customer_churn (
customer_id VARCHAR(20),
gender VARCHAR(10),
age INT,
married VARCHAR(10),
number_of_dependents INT,
city VARCHAR(100),
zip_code VARCHAR(10),
latitude DECIMAL(10,6),
longitude DECIMAL(10,6),
number_of_referrals INT,
tenure_in_months INT,
offer VARCHAR(20),
phone_service VARCHAR(10),
avg_monthly_long_distance_charges DECIMAL(10,2),
multiple_lines VARCHAR(10),
```

```
internet_service VARCHAR(10),
internet_type VARCHAR(30),
avg_monthly_gb_download INT,
online_security VARCHAR(10),
online_backup VARCHAR(10),
device_protection_plan VARCHAR(10),
premium_tech_support VARCHAR(10),
streaming_tv VARCHAR(10),
streaming_movies VARCHAR(10),
streaming_music VARCHAR(10),
unlimited_data VARCHAR(10),
contract VARCHAR(30),
paperless_billing VARCHAR(10),
payment_method VARCHAR(50),
monthly_charge DECIMAL(10,2),
total_charges DECIMAL(12,2),
total_refunds DECIMAL(12,2),
total_extra_data_charges DECIMAL(12,2),
total_long_distance_charges DECIMAL(12,2),
total_revenue DECIMAL(12,2),
customer_status VARCHAR(20),
churn_category VARCHAR(50),
churn_reason VARCHAR(255)
);
```

Created tables customer_churn and zip_population

```
COPY stg_customer_churn
FROM
's3://telecom-redshift-assignment-ayush/raw/telecom_customer_churn.csv'
IAM_ROLE 'arn:aws:iam::890615325018:role/RedshiftS3AccessRole'
CSV
IGNOREHEADER 1;
```

Copied the customer churn data from the s3 bucket . injected the data from s3 bucket to dev database in redshift query editor v2.
Ignoring the header because attributes is already created .

```
COPY stg_zip_population
FROM
's3://telecom-redshift-assignment-ayush/raw/telecom_zipcode_population.csv
'
IAM_ROLE 'arn:aws:iam::890615325018:role/RedshiftS3AccessRole'
CSV
IGNOREHEADER 1;
```

Done same for zip_population :

Copy command -> then table name -> from -> then url of the file location in s3 -> then iam_role -> arn id and then csv -> ignore header = 1 because attribute is already created with the table.

```
SELECT COUNT(*) FROM stg_customer_churn;

SELECT COUNT(*) FROM stg_zip_population;
```

Calculated the count of data in both the tables customer_churn have 7043 count and zip_population have 1671 count .

Task 6 — Create Analytical Table

Create a final analytical table by joining customer churn data with the zip code population dataset.
The final table should include:

customer_id city
zip_code
population
tenure
monthly_charges
total_charges
customer_status

Use appropriate DISTKEY and SORTKEY choices

DISTKEY Choice

Join happens on:

c.zip_code = p.zip_code

Therefore:

DISTKEY(zip_code)

is the correct answer.

SORTKEY Choice

Most analysis will use:

customer_status
tenure_in_months

Examples:

- churn rate
- churn by tenure
- churned customers

Therefore:

SORTKEY(customer_status, tenure_in_months)

is a reasonable choice

Why Do We Need DISTKEY and SORTKEY?

Imagine you have:

1 Billion Customer Records

stored across multiple Redshift nodes.

When you run a query:

```
SELECT *
FROM customers c
JOIN zip_population p
ON c.zip_code = p.zip_code;
```

Redshift needs to find matching rows quickly.

That's where:

DISTKEY → Data Distribution
SORTKEY → Data Ordering

come into play.

DISTKEY (Distribution Key)

What does it do?

DISTKEY decides:

Which node stores which rows

Imagine you have 4 nodes:

Node 1
Node 2
Node 3
Node 4

and this data:

Zip Code
90001
90002
90001
90003

If:

DISTKEY(zip_code)

then Redshift tries to keep rows with the same zip code on the same node.

Example:

Node 1
90001
90001

Node 2
90002

Node 3
90003

Why is this useful?

Suppose:

Customer Table

Zip Code
90001
90002

Population Table

Zip Code
90001
90002

and both use:

DISTKEY(zip_code)

Now:

Customer 90001

Population 90001

are on the same node.

Redshift doesn't need to move data across the network.

This makes joins much faster.

SORTKEY

Now let's understand SORTKEY.

What does SORTKEY do?

SORTKEY decides:

How data is physically ordered on disk.

Suppose you have:

Customer Status
Stayed
Churned
Stayed
Churned

Without sorting:

Stayed
Churned
Stayed
Churned
Stayed
Churned

Everything is mixed.

With:

SORTKEY(customer_status)

Redshift stores:

Churned
Churned
Churned
Stayed
Stayed
Stayed

Why is this useful?

Suppose query:

```
SELECT *
FROM customer_churn_analytics
WHERE customer_status='Churned';
```

Without SORTKEY:

Scan entire table

With SORTKEY:

Scan only churned blocks

Much faster.

```sql
CREATE TABLE customer_churn_analytics
DISTKEY(zip_code)
SORTKEY(customer_status, tenure_in_months)
AS
SELECT
    c.customer_id,
    c.city,
    c.zip_code,
    p.population,
    c.tenure_in_months,
    c.monthly_charge,
    c.total_charges,
    c.customer_status
FROM stg_customer_churn c
LEFT JOIN stg_zip_population p
ON c.zip_code = p.zip_code;
```

```sql
SELECT COUNT(*)
FROM customer_churn_analytics;
```

Output is 7043

Task 7 — Perform Data Analysis Using the final analytical table, generate the following outputs:
1. Churn rate across all customers
2. Top cities with the highest number of churned customers
3. Customer churn distribution by tenure group
4. Total revenue lost due to churn
5. Population vs customer count by zip code Store the SQL scripts used for generating these results.

Query 1: Churn Rate Across All Customers
Expected output : Total_customers , churned_customers ,churn_rate_percentage

```sql
SELECT
    COUNT(*) AS total_customers,
    SUM(
        CASE
            WHEN customer_status = 'Churned' THEN 1
            ELSE 0
        END
    ) AS churned_customers,
    ROUND(
        100.0 *
        SUM(
            CASE
                WHEN customer_status = 'Churned' THEN 1
                ELSE 0
            END
        ) / COUNT(*),
        2
    ) AS churn_rate_percentage
FROM customer_churn_analytics;
```

Query 2: Top Cities With Highest Number of Churned Customers

```sql
SELECT
    city,
    COUNT(*) AS churned_customers
FROM customer_churn_analytics
WHERE customer_status = 'Churned'
GROUP BY city
ORDER BY churned_customers DESC
LIMIT 10;
```

Query 3: Customer Churn Distribution By Tenure Group

```sql

```

```sql
SELECT
    CASE
        WHEN tenure_in_months BETWEEN 0 AND 12 THEN '0-12 Months'
        WHEN tenure_in_months BETWEEN 13 AND 24 THEN '13-24 Months'
        WHEN tenure_in_months BETWEEN 25 AND 48 THEN '25-48 Months'
        ELSE '49+ Months'
    END AS tenure_group,
    COUNT(*) AS churned_customers
FROM customer_churn_analytics
WHERE customer_status = 'Churned'
GROUP BY 1
ORDER BY 1;
```

Query 4: Total Revenue Lost Due To Churn

```sql
SELECT
    ROUND(SUM(total_revenue),2) AS revenue_lost_due_to_churn
FROM stg_customer_churn
WHERE customer_status = 'Churned';
```

Query 5: Population vs Customer Count By Zip Code

```sql
SELECT
    zip_code,
    population,
    COUNT(customer_id) AS customer_count
FROM customer_churn_analytics
GROUP BY
    zip_code,
    population
ORDER BY customer_count DESC
LIMIT 20;
```

Task 8 — Redshift Processing and Optimization
Perform the following Redshift maintenance operations**:**
ANALYZE
VACUUM
Explain briefly how they improve Redshift performance

What is ANALYZE?

Redshift maintains statistics about your tables.

Example:

How many rows?
How many unique values?
Data distribution?

The query optimizer uses these statistics to create efficient execution plans.

Run ANALYZE

```
ANALYZE customer_churn_analytics;
```

You can also run:

```
ANALYZE stg_customer_churn;
ANALYZE stg_zip_population;
```

Interview Explanation

> **ANALYZE updates table statistics used by the Redshift query optimizer. Accurate statistics help Redshift choose efficient execution plans, resulting in faster query performance.**

What is VACUUM?

Over time:

DELETE
UPDATE
INSERT

operations create fragmented storage.

VACUUM:

Reclaims space
Re-sorts data
Improves scan performance

Run VACUUM

VACUUM customer_churn_analytics;

Interview Explanation

> **VACUUM reorganizes and sorts table data, removes fragmentation, and reclaims unused storage space. This improves query performance and storage efficiency.**

```
ANALYZE customer_churn_analytics;

ANALYZE stg_customer_churn;
ANALYZE stg_zip_population;


VACUUM customer_churn_analytics;
```

Task 9 — Clean Up Resources After completing the assignment:

- **Delete or pause the Redshift cluster**
- **Ensure S3 storage is minimal**